

GenAI-Powered Data Certification Engine

YouTube Video Presentation Script

Opening — The Problem

Imagine you are a data engineer in a large enterprise.

Every day, new datasets arrive from different systems — banking transactions, customer data, sales data, operational systems, or external partners.

Before that data can be trusted for analytics, reporting, machine learning, or regulatory processes, someone needs to answer a simple question:

Can we certify this data as trustworthy?

Traditionally, this process involves a series of manual checks.

Are all required columns present?

Are there too many null values?

Are primary keys duplicated?

Has the data distribution changed?

Has personally identifiable information been properly masked?

Is the data stored in an approved region?

And has the data exceeded its retention period?

The problem is that these checks are often implemented as disconnected scripts, dashboards, and manual reviews.

So today, I want to show you a different approach:

A GenAI-powered Data Certification Engine.

The idea is simple:

Instead of treating data certification as a collection of isolated validation scripts, we build an intelligent certification workflow that combines deterministic data-quality checks, machine-learning-based drift detection, compliance validation, and GenAI-generated audit summaries.

What Are We Building?

The implementation for this demo is contained in:

`data_certification_agents.py`

At a high level, our certification engine performs four major activities:

Completeness → **Drift Detection** → **Compliance** → **GenAI Certification Summary**

And finally, it produces a certification decision:

PASS, CONDITIONAL, or FAIL.

So let's walk through each layer.

1. Data Completeness

The first question is:

Is the dataset structurally and statistically complete enough to be trusted?

Our engine performs three important checks.

Required Columns

First, we validate whether all mandatory columns are present.

For example, imagine our customer dataset requires:

- `customer_id`
- `transaction_id`
- `transaction_amount`
- `transaction_date`
- `region`

If one of these columns is missing, the dataset should not automatically be certified.

This is a simple check, but extremely important.

Because downstream systems often assume that these fields exist.

Null-Rate Validation

Next, we look at missing values.

Suppose our business policy says:

Customer ID must have less than 1% null values.

If today's dataset contains 8% missing customer IDs, the data may technically load successfully — but it shouldn't be certified.

This is where certification becomes more than just a technical pipeline check.

*We are evaluating the dataset against **business-defined quality policies**.*

Duplicate Primary Keys

Finally, we check for duplicate primary keys.

For example:

customer_id = 1001

should normally identify one unique customer.

If that identifier appears multiple times when uniqueness is expected, it could indicate:

- *duplicate ingestion*
- *incorrect joins*
- *upstream processing issues*
- *replayed events*
- *or data corruption*

Our engine detects these violations and incorporates them into the overall certification score.

2. Drift Detection

Now we move to a more interesting part of the architecture.

Data quality isn't only about missing values.

A dataset can be perfectly complete and still be wrong.

Imagine yesterday's transaction amounts looked like this:

\$20

\$35

\$50
\$75
\$100

And today, suddenly we see:

\$500
\$800
\$1,200
\$2,000

There may be no null values.

There may be no duplicate IDs.

All required columns are present.

But something has fundamentally changed.

*This is called **data drift**.*

And that's why our certification engine includes PyTorch-based drift detection.

Mean-Shift Detection

The first technique looks at numerical features and compares their statistical behavior between the baseline dataset and the current dataset.

For example:

Baseline average transaction amount: \$85

Current average transaction amount: \$240

That represents a significant shift.

The engine can calculate this difference and compare it against a configured sensitivity threshold.

This gives us a simple but powerful signal:

Has the behavior of the data changed?

Autoencoder-Based Drift Detection

But we can go one step further.

The implementation also uses an autoencoder-based approach.

An autoencoder learns to reconstruct data patterns.

We can train or establish a baseline reconstruction behavior and then compare it with the current dataset.

If the current data produces significantly higher reconstruction error, that can indicate that the current dataset no longer resembles the baseline distribution.

In simple terms:

The model has learned what normal data looks like.

When new data behaves very differently, the reconstruction error increases.

The engine then compares the reconstruction behavior between baseline and current data.

This gives us another independent drift signal.

And this is where machine learning becomes part of the certification process.

3. Compliance Validation

Data quality is only one side of certification.

In enterprise environments, we also need to ask:

Is this data compliant with organizational policies?

Our engine includes three compliance checks.

PII Masking

First:

Has sensitive information been properly masked?

For example:

Customer email addresses, phone numbers, national identifiers, or other sensitive fields may need to be masked before the dataset is certified.

The certification engine checks the configured PII columns against the expected masking policy.

This is particularly important when data moves from production systems into analytics, testing, or AI environments.

Region Allowlist

Next:

Is the data being processed or stored in an approved region?

For example, an organization may allow data processing only in:

US

EU

or specific approved geographic regions.

If a dataset appears in a region that isn't on the organization's allowlist, the certification should fail or become conditional.

This is a very simple policy concept, but it can have significant regulatory implications.

Retention Window

The third check is retention.

Every organization can define policies such as:

Transaction data may be retained for 365 days.

If the current dataset contains records older than the configured retention period, the engine flags the violation.

So now our certification isn't only asking:

"Is this data good?"

It is also asking:

"Is this data allowed?"

4. Bringing GenAI Into the Process

Now we arrive at the GenAI component.

Once all the deterministic and machine-learning checks are complete, we have a lot of information.

For example:

Completeness:

PASS

Drift:

WARNING

Compliance:

FAIL

Overall score:

72

Decision:

CONDITIONAL

But showing these numbers to an auditor or business stakeholder isn't always enough.

This is where GenAI becomes useful.

The engine can send the certification results to Mistral AI using:

mistral-large-latest

*when a **MISTRAL_API_KEY** is configured.*

The model converts the structured certification results into a human-readable audit summary.

Instead of an auditor having to interpret dozens of fields, they can receive a concise explanation such as:

The dataset passed structural completeness checks but exhibited significant distribution drift in transaction-related features. Compliance validation identified a PII masking violation. Certification is therefore conditional pending remediation.

That is much easier for humans to consume.

Why Use GenAI Here?

An important architectural principle is:

GenAI should not make the certification decision.

This is critical.

We don't want an LLM deciding:

"I think this dataset looks safe."

Instead, deterministic rules and statistical/ML checks produce the actual evidence.

*The GenAI model is used to **explain the evidence**.*

So the architecture becomes:

**Data → Deterministic Checks → ML Checks → Compliance Rules → Score → Decision
→ GenAI Explanation**

This is a much safer enterprise pattern.

Deterministic Fallback

There is another interesting design decision in this implementation.

What happens if Mistral isn't available?

Maybe the API key isn't configured.

Maybe there is a network problem.

Maybe the dependency isn't installed.

Should the certification pipeline stop?

No.

The engine falls back to a deterministic local summary.

This is an important production engineering principle:

The GenAI layer should enhance the workflow, not become a single point of failure.

The actual certification decision remains independent of the LLM.

5. Certification Scoring

Now let's talk about the final decision.

The engine combines the different validation categories using configurable weights.

For example, we might assign weights to:

- *Completeness*
- *Drift*
- *Compliance*

Each category contributes to the overall certification score.

Then we apply configurable decision thresholds.

The result can be:

PASS

The dataset meets the certification requirements.

CONDITIONAL

The dataset has issues that require attention but may still be usable under defined conditions.

FAIL

The dataset violates critical quality or compliance requirements.

This is important because enterprise certification is rarely a simple binary problem.

Sometimes the right answer isn't simply:

"Good data" or "Bad data."

Sometimes it is:

"Good enough, but with conditions."

6. The End-to-End Flow

Let's visualize the entire workflow.

A new dataset arrives.

↓

The certification engine loads the baseline and current datasets.

↓

First, we perform completeness validation.

Are the required columns available?

Are null rates within acceptable limits?

Are primary keys unique?

↓

Next, we perform drift detection.

Has the numerical distribution changed?

Does the current dataset behave differently from the baseline?

↓

Then we perform compliance validation.

Is PII properly masked?

Is the region allowed?

Are retention policies satisfied?

↓

All these results are combined into a weighted certification score.

↓

The engine produces:

PASS / CONDITIONAL / FAIL

↓

Finally, GenAI converts the structured evidence into a human-readable audit summary.

↓

And the complete certification report becomes an **audit-ready artifact**.

7. What the Demo Generates

When we run:

```
python data_certification_agents.py
```

the demo generates baseline and current datasets.

The current dataset intentionally contains quality, compliance, and drift issues.

That's important because we don't want a demo that always produces:

PASS

We want to see what happens when the system detects problems.

The resulting JSON report contains information such as:

- Run ID
- UTC timestamp
- Certification decision
- Overall weighted score
- Completeness results
- Drift results
- Compliance results
- Audit summary

This gives us both:

Machine-readable evidence

and

Human-readable explanation.

8. Why This Architecture Matters

Now let's step back from the Python implementation.

The bigger idea isn't the script.

The bigger idea is the architecture.

Traditional data certification often looks like:

Dataset → Manual Checks → Spreadsheet → Human Review → Certification

Our architecture changes that to:

Dataset → Automated Validation → ML Drift Detection → Compliance Policy → Scoring → GenAI Explanation → Audit Report

This provides several benefits.

Automation

Certification can run automatically whenever a new dataset arrives.

Consistency

The same policies are applied every time.

Early Detection

Data drift can be detected before it impacts downstream analytics or machine-learning models.

Compliance

Regulatory and organizational policies become executable rules.

Explainability

GenAI converts technical validation results into a summary that business and audit teams can understand.

Auditability

Every certification run can have a unique run ID, timestamp, decision, score, and detailed evidence.

9. Where Agents Fit In

There is another important architectural evolution we can make.

Instead of implementing everything as one large script, we can decompose the workflow into specialized agents.

For example:

Certification Orchestrator

↓

Completeness Agent

Drift Detection Agent

Compliance Agent

↓

Scoring Agent

↓

Audit Summary Agent

The orchestrator coordinates the workflow.

Each specialized agent focuses on one responsibility.

For example:

The Completeness Agent answers:

"Is the dataset structurally complete?"

The Drift Agent answers:

"Has the dataset behavior changed?"

The Compliance Agent answers:

"Does the dataset comply with policy?"

The Scoring Agent answers:

"What is the certification decision?"

And finally, the GenAI Audit Agent answers:

"How do we explain this decision to a human?"

*This is where the architecture starts becoming truly **agentic**.*

10. Enterprise Evolution

The demo is intentionally lightweight, but the same architecture can evolve into a production platform.

Imagine integrating it with:

- *AWS S3*
- *Snowflake*
- *Databricks*
- *Apache Spark*
- *Airflow*
- *Kafka*
- *Data catalogs*
- *Governance platforms*
- *Enterprise IAM*
- *SIEM systems*

Now the workflow could become:

Data Arrival

↓

Certification Trigger

↓

Automated Quality Checks

↓

ML Drift Detection

↓

Compliance Validation

↓

Certification Decision

↓

GenAI Audit Explanation

↓

Data Catalog Update

↓

Notification / Workflow

If certification passes, the dataset becomes available to downstream consumers.

If certification fails, the pipeline can automatically quarantine the dataset and notify the data owner.

That is where this concept becomes much more than a Python script.

*It becomes a **Data Trust Platform**.*

Closing

So, the key takeaway from this project is this:

Data certification doesn't have to be a manual checklist.

We can combine:

Deterministic rules

with

Machine learning

with

Compliance policies

with

Generative AI

to build an automated, explainable, and audit-ready certification engine.

And perhaps the most important design principle is this:

Let deterministic systems decide. Let AI explain.

That separation gives us the benefits of GenAI without allowing an LLM to become the source of truth for critical data-governance decisions.

The complete implementation is available in the [AISamples](#) repository under:

[Chapter_AgentsForDataCertification](#)

You can clone the repository, install the required Python dependencies, and run the certification engine locally.

If you found this architecture useful, consider subscribing to the channel.

In the next step, we can take this prototype further and turn it into a fully agentic architecture with specialized agents for quality, drift, compliance, scoring, and audit reporting.

Thanks for watching, and I'll see you in the next video.